

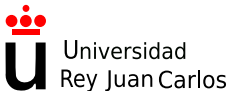
Laboratorio de Tecnologías Audiovisuales en la Web (2025-26)

Grado en Ingeniería en Sistemas Audiovisuales y Multimedia (URJC)

Jesús M. González Barahona, Gregorio Robles, David Moreno,
Alberto Rodríguez, Alberto García, Sergio Montes

<http://cursosweb.github.io>
GSyC, Universidad Rey Juan Carlos

9 de marzo de 2026



- 1 Presentación de la asignatura
- 2 Cookies HTTP
- 3 REST: Representational State Transfer
- 4 Arquitectura modelo-vista-controlador
- 5 Introducción a XML
- 6 Ajax y tecnologías relacionadas
- 7 Prácticas: Introducción a Python
 - Material principal
 - Material adicional
- 8 Prácticas: Aplicaciones Web
- 9 Prácticas: Introducción a Django
- 10 Hojas de estilo CSS
- 11 Bootstrap
- 5 Créditos, licencia...

Presentación de la asignatura

Datos, datos, datos

- Profesores:
 - Jesús M. González Barahona (jesus.gonzalez @ urjc.es)
 - David Moreno Lumbreras (david.morenolu @ urjc.es)
 - Gregorio Robles (grex @ gsync.urjc.es)
 - Juan González Gómez (juan.gonzalez.gomez @ urjc.es)
- Grupo de Sistemas y Comunicaciones (GSyC)
- Horario: L (11:00-13:00) y ; (13:00-15:00)
- Tutoría: Jueves 17:00-19:00 (via videoconferencia)

Clases: Laboratorios III, L2103

Campus virtual: <http://aulavirtual.urjc.es>

Cursos web: <http://cursosweb.github.io>

GitLab de la EIF: <http://gitlab.eif.urjc.es>

¿De qué va todo esto?



Entendiendo cómo funciona la web

En concreto...

- Cómo se construyen los sistemas reales que se usan en Internet
- Qué tecnologías se están usando
- Qué esquemas de seguridad hay
- Cómo encajan las piezas
- En la medida de lo posible, “manos en la masa”

Ejemplos

- ¿Qué es una aplicación web?
- ¿Qué es una sesión?
- ¿Cómo construir un servicio REST?
- Acaba con la magia de los servicios web
- ¿Cómo se hace un servicio basado en contenidos?

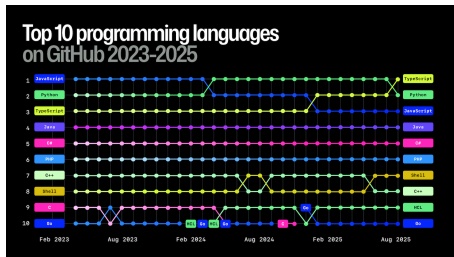
Fundamentos de la asignatura

A person with short dark hair, seen from the back, is sitting at a desk. They are wearing a blue long-sleeved shirt. In front of them are two computer monitors. The larger monitor in the background displays a code editor with syntax-highlighted code on a dark background. The smaller laptop monitor in the foreground shows a blue-toned image of a guitar. The person's right hand is visible near the laptop. The scene is dimly lit, with a desk lamp visible on the right side of the frame.

**La programación es el
lenguaje de la tecnología**

Lenguaje de Programación: Python

Jan 2020	Jan 2019	Change	Programming Language	Rating	Change
1	1		Java	16.896%	-0.01%
2	2		C	15.773%	+2.48%
3	3		Python	9.704%	+1.41%
4	4		C++	5.574%	-2.58%
5	7	▲	C#	5.349%	+2.07%
6	5	▼	Visual Basic .NET	5.287%	-1.17%
7	6	▼	JavaScript	2.451%	-0.85%
8	8		PHP	2.405%	-0.28%
9	15	▲	Swift	1.795%	+0.61%
10	9	▼	SQL	1.504%	-0.77%
We use cookies to analyse our traffic and to show ads. By using our website, you agree to our use of cookies.					
13	10	▼	Objective-C	0.929%	-0.85%
14	16	▲	Go	0.900%	-0.22%
15	14	▼	Assembly language	0.877%	-0.32%
16	20	▲	Visual Basic	0.831%	-0.20%
17	25	▲	D	0.825%	+0.25%
18	12	▼	R	0.808%	-0.52%
19	13	▼	Perl	0.746%	-0.48%
20	11	▼	MATLAB	0.737%	-0.76%



Primer Mandamiento:
Amarás Python por encima de (casi) todo.
TIOBE (enero 2020) GitHub Octoverse (octubre 2025)

<https://www.youtube.com/watch?v=UNSoPa-XQN0>

Plataforma: Django



DESARROLLO WEB

Primera Generación

HTML

CGI

Segunda Generación

PHP

JSP

PERL

Tercera Generación

RAILS

DJANGO

SYMFONY

Segundo Mandamiento:

No tomarás el nombre de Django en vano.

Plataforma: Django (2)

Algunas referencias a principales marcos de desarrollo web (2025):

- “10 Software Development Frameworks That Will Dominate 2025”
<https://www.index.dev/blog/10-programming-frameworks>
- “Web Application Development | Top 10 Frameworks in 2025”
<https://www.sencha.com/blog/web-application-development-top-frameworks/>
- “The Top 10 Frameworks for Web Development in 2025”
<https://medium.com/@technosoftware21/the-top-10-frameworks-for-web-development-in-2025-4a7fc54f47dc>

Suele ser el preferido en Python, y uno de los 10 principales.

Metodología

Objetivo principal:
conceptos básicos de construcción de sitios web modernos

- Clases de teoría y de prácticas, pero...
- Teoría en prácticas, prácticas en teoría
- Uso de resolución de problemas para aprender
- Construcción de un proyecto a lo largo del curso

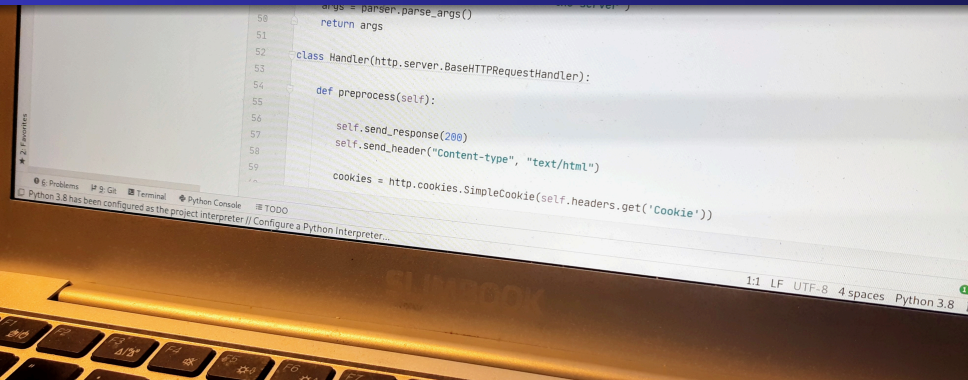
Fundamentalmente, entender lo fundamental

Fundamentos de la asignatura



Aprender no puede ser aburrido

Fundamentos de la asignatura (2)



La letra con código entra

Fundamentos de la asignatura (3)

La IA generativa está aquí
para quedarse

Las Clases

- Empezamos en punto
- A veces: 5–10 min. de Frikiminutos
 - Gadgets tecnológicos
 - Aplicaciones
 - Cuestiones interesantes
 - ...
- A veces, explicación de los conceptos más importantes y luego realización de ejercicios
- A veces, al revés
- Ejercicios para hacer fuera de clase (y entregar)

Fundamentos de la asignatura

El estudiante es el centro del
aprendizaje



Laboratorio



- Laboratorios Linux de la EIF
- Este curso, a distancia: vía VNC, vía ssh...
- <https://labs.eif.urjc.es>

Software (fundamental) que utilizaremos:

- Ubuntu
- Python, Django
- PyCharm

Evaluación

- Microprácticas diarias (entrega foro/GitLab): 0 a 1
- Miniprácticas preparatorias: 0 a 1
- Proyecto final (obligatorio): 0 a 2.
- Opciones y mejoras práctica final: 0 a 3
- Teoría (obligatorio): 0 a 4.
 - Ojo: teoría fuertemente relacionada con prácticas
- Nota final: Suma de notas, moderada por la interpretación del profesor
- Mínimo para aprobar:
 - Aprobado en teoría (2) y proyecto final (1), y
 - 5 puntos de nota final en total

Evaluación (2)

- Evaluación teoría: prueba escrita (quizás en ordenador)
- Microprácticas diarias y miniprácticas incrementales:
 - es muy recomendable hacerlas
- Evaluación proyecto final
 - posibilidad de examen presencial para proyecto final
 - ¡tiene que funcionar en el laboratorio!
 - enunciado mínimo obligatorio supone 1, se llega a 2 sólo con calidad y cuidado en los detalles
- Opciones y mejoras proyecto final:
 - permiten subir mucho la nota
- Evaluación extraordinaria:
 - prueba escrita (si no se aprobó la ordinaria)
 - nuevo proyecto final (si no se aprobó el ordinario)

Proyecto final

Ejemplos del pasado:

- Servicio de búsqueda de hoteles
- Servicio de apoyo a la docencia
- Sitio de intercambio de fotos
- Aplicación web de autoevaluación docente
- Agregador de blogs (canales RSS)
- Agregador de microblogs (Identi.ca, Twitter)

Proyecto final

Este curso...

■ ■ ■ ■

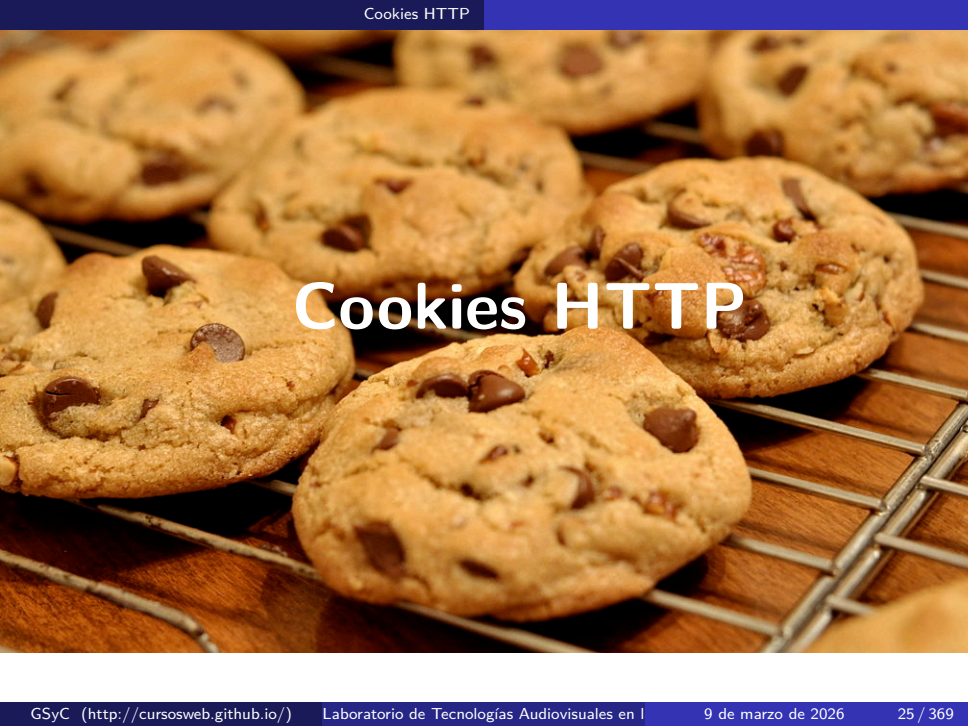
Ejemplos de prácticas finales de otros años

- Niki Montero (2024):
<https://www.youtube.com/watch?v=coAf8o0gQ00>
- Ainhoa Hernández (2021):
<https://www.youtube.com/watch?v=7Bd33IS3XBE>
- Noelia López (2019):
<https://youtube.com/watch?v=VRg37vwU110>
- Fernando Yustas (2014):
<https://youtube.com/watch?v=TUUMVEaBzeg>

(puedes buscar en YouTube muchos más ejemplos)

Aquí se enseñan cómo son las cosas
que se usan en el mundo real

Las buenas noticias son...
que no son tan difíciles



Cookies HTTP

Cookies

- Forma de mantener estado en un protocolo sin estado (HTTP)
- Forma de almacenar datos en el lado del cliente
- Dos versiones principales:
 - Versión 0 (Netscape)
 - Version 1 (RFC 2109 / RFC 2965)

Cabeceras HTTP para cookies (version 0)

- Set-Cookie: De servidor a navegador

```
Set-Cookie: statusmessages="deleted"; Path=/  
Expires=Wed, 31-Dec-97 23:59:59 GMT
```

- Nombre de la cookie: statusmessages
- Valor: "deleted"
- Path: / (todo el sitio del que se recibió)
- Expira: 31-Dec-97 23:59:59 GMT

- Cookie: De navegador a servidor

```
Cookie: statusmessages="deleted"
```

- Nombre de la cookie: statusmessages
- Valor: "deleted"

RFC 2965 (version 1) tiene: "Cookie", "Cookie2" y "Set-Cookie2"

Estructura de Set-Cookie

- Nombre y valor (obligatorios)
 - Uso normal: "nombre=valor"
 - También valor nulo: "nombre="
- Fecha de expiración
 - "Expires=fecha"
 - Si no tiene, cookie no persistente (sólo en memoria)
- Path: camino para el que es válida
 - "Path=/camino"
 - Prefijo de caminos válidos
- Domain: dominio para el que es válida
 - El servidor ha de estar en el dominio indicado
 - Si no se indica, servidor que sirve la cookie
- Seguridad: se necesita SSL para enviar la cookie
 - Campo "Secure"
- Campos separados por ";

Estructura de Cookie

- Lista de pares nombre - valor
- Cada par corresponde a un “Set-Cookie”
- Se envían las cookies válidas para el dominio y el path de la petición HTTP
- Si no se especificó dominio en “Set-Cookie”, el del servidor
- Si no se especificó camino en “Set-Cookie”, todo

Cookie: user=jgb; last=5; edited=False

Límites para las cookies

- Originalmente: 20 cookies del mismo dominio
- La mayoría de los navegadores: 30 o 50 cookies del mismo dominio
- Cada cookie: como mucho 4 Kbytes

Gestión de sesión en HTTP

- Mediante cookies: normalmente, identificador de sesión en la cookie

Set-Cookie: session=ab34cd-34fd3a-ef2365

- Reescritura de urls: se añade identificador a la url

`http://sitio.com/path;session=ab34cd-34fd3a-ef2365`

- Campos escondidos en formularios HTML

```
<form method="post" action="http://sitio.com/path">  
  <input type="hidden" name="session" value="ab34cd-34fd3a-ef2365">  
  ...  
  <input type="submit">  
</form>
```

Referencias

- Persistent Client State HTTP Cookies
(especificación original de Netscape)
http://curl.haxx.se/rfc/cookie_spec.html
- RFC 2109: HTTP State Management Mechanism
<http://tools.ietf.org/html/rfc2109>
- RFC 2965: HTTP State Management Mechanism
<http://tools.ietf.org/html/rfc2965>

REST: Representational State Transfer

REST: El estilo arquitectural de la web

- REST: REpresentational State Transfer
“Conjunto de recursos web (máquina de estados), en el que se realizan acciones seleccionando recursos a los que se les aplican operaciones (transiciones de estado), obteniendo representaciones de ese estado (que recibe el navegador).”
- Estilo arquitectural para sistemas distribuidos
 - Ampliamente extendido en la web estática (ficheros y directorios)
 - Parcialmente extendido en la web programática (aplicaciones y servicios web)

Principios REST

- Cliente (interfaz de usuario) - servidor (lógica de negocio, datos)
- Sin estado: cada petición lleva todo su contexto
- Cacheable: cada respuesta es marcada como cacheable, o no
- Interfaz uniforme (recurso, urls, métodos, representaciones...)
- Sistema por capas: cada componente sólo “ve” la capa que invoca
- Código en el cliente (sobre todo, para interfaz de usuario)

Interfaz: Cada recurso debe tener una URL

Un recurso puede ser cualquier tipo de información que quiere hacerse visible en la web: documentos, imágenes, servicios, gente

```
http://example.com/profiles
```

```
http://example.com/profiles/juan
```

```
http://example.com/posts/2007/10/11/ventajas_rest
```

```
http://example.com/posts?f=2007/10/11&t=2007/11/17
```

```
http://example.com/shop/4554/
```

```
http://example.com/shop/4554/products
```

```
http://example.com/shop/4554/products/15
```

```
http://example.com/orders/juan/15
```

Interfaz: Cada recurso debe tener una URL

Dos tipos de recursos:

- Colecciones, como `http://example.com/resources/`
- Elementos, como `http://example.com/resources/142`

Los métodos tienen un significado ligeramente diferente se trate de una colección o un elemento.

Interfaz: Los recursos tienen hiperenlaces a otros recursos

Los formatos más usados (XHTML, Atom, ...) ya lo permiten. También aplica a los formatos inventados por nosotros mismos

```
<profiles self="http://example.com/profiles">
  <profile type="moderator"
    self="http://example.com/profiles/123">
    <name>Pepito Pérez</name>
    <company ref="http://example.com/companies/321">
      Compañía XYZ</company>
    </profile>
  ...
</profiles>
```

Interfaz: Conjunto limitado y estándar de métodos

- GET: Obtener información (quizá de caché)
 - No cambia estado, idempotente
- PUT: Actualizar o crear un recurso conocida su URL
 - Cambia estado, idempotente
- POST: Crear un recurso conociendo la URL de un constructor de recursos
 - Cambia estado, NO idempotente
- DELETE: Borrar un recurso conocida su URL
 - Cambia estado, idempotente

Interfaz: Múltiples representaciones por recurso

Las representaciones muestran el estado actual del recurso en un formato dado

```
GET /profile/1234  
Host: example.com  
Accept: text/html
```

```
GET /profile/1234  
Host: example.com  
Accept: text/x-vcard
```

```
GET /profile/1234  
Host: example.com  
Accept: application/mi-vocabulario-propio+xml
```


Interfaz: Comunicación sin estado

- En REST se mantiene estado, pero sólo en los recursos, no en la aplicación (o sesión).
- Una petición del cliente al servidor debe contener todo el contexto necesario (no puede basarse en información previa asociada a la comunicación).
- No hay datos que permitan identificar sesión en el servidor. El servidor sólo guarda y maneja el estado de los recursos que aloja.
- El cliente puede guardar identificación de sesión (y enviárselo al servidor), por ejemplo en forma de cookies.
- Esto permite escalabilidad (servidores más sencillos). Implica mayor ancho de banda, pero facilita las *cachés*.

REST: Ventajas de REST

- Maximiza la reutilización
 - Todos los recursos tienen identificadores = hacen más grande la Web
 - La visibilidad de los recursos que forman una aplicación/servicio permite usos no previstos
- Minimiza el acoplamiento y permite la evolución
 - La interfaz uniforme esconde detalles de implementación
 - El uso de hipertexto permite que sólo la primera URL usada para acceder acople un sistema a otro
- Elimina condiciones de fallo parcial
 - Fallo del servidor no afecta al cliente
 - El estado siempre puede ser recuperado como un recurso
- Escalado sin límites
 - Los servicios pueden ser replicados en cluster, cacheados y ayudados por sistemas intermediarios
- Unifica los mundos de las aplicaciones web y servicios web. El mismo código puede servir para los dos fines.

Funcionalidades de HTTP no tan conocidas

- Códigos de respuesta estandarizados: los entiende un navegador, los proxies, o nuestras propias aplicaciones
 - Information: 1xx, Success 2xx, Redirection 3xx, Client Error 4xx, Server Error 5xx
- Negociación de contenido: la misma URL puede mandar la representación más adecuada al cliente (HTML, Atom, texto)
- Redirecciones
- Cacheado (incluyendo mecanismos para especificar el periodo de validez y expiraciones)
 - Dos tipos: browser, proxy
 - Cabecera HTTP: Cache-Control

Funcionalidades de HTTP no tan conocidas (2)

- Compresión
- División de la respuesta en partes (Chunking)
- GET condicional (sólo se envía la respuesta si ha habido cambios)
 - Permite ahorrar ancho de banda, procesado en el cliente y (posiblemente) procesado en el servidor
 - Dos mecanismos:
 - ETag y If-None-Match: identificador asociado al estado de un recurso.
Ejemplo: hash de la representación en un formato dado
 - Last-Modified y If-Modified-Since: fecha de última actualización

Ejemplo de interacción HTTP: Petición

GET / HTTP/1.1

Host: www.ejemplo.com

User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1;
en-US; rv:1.8.1.3)...

Accept: text/xml,application/xml,application/xhtml+xml,
text/html;q=0.9...

Accept-Language: en-us,en;q=0.5

Accept-Encoding: gzip,deflate

Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7

If-None-Match: "4c083-268-423f1dc5"

If-Modified-Since: "4c083-268-423f1dc5"

Keep-Alive: 300

Connection: keep-alive

Cookie: [...]

Cache-Control: max-age=0

Ejemplo de interacción HTTP: Respuesta

HTTP/1.x 200 OK

Date: Thu, 17 May 2007 14:06:30 GMT

Server: Microsoft-IIS/6.0

ETag: "4c083-268-423f1dc6"

Last-Modified: Mon, 21 Mar 2005 19:17:26 GMT

Cache-Control: private

Content-Type: text/html; charset=utf-8

Content-Length: 16850

Mecanismos de seguridad

- Autenticación:
 - HTTP Basic
 - HTTP Digest
 - OAuth
- Importante: los mecanismos de autenticación de HTTP son extensibles
- Cifrado y firma digital: SSL

Límites actuales de REST en la web

- Obliga a los desarrolladores a pensar diferente
- Carencia de soporte de PUT y DELETE en navegadores y cortado en cortafuegos
- Mecanismos de seguridad limitados (en comparación con WS-Security): está mejorando rápidamente
- La negociación de contenidos no funciona del todo bien en la práctica
- Algunos piden un método más: PATCH
- Poca documentación

Ejemplos RESTful

- En la web
 - Todos los sitios web estáticos
 - Servicios web de sólo lectura
 - Servicios web Google (GData)
 - Aplicaciones web hechas con Ruby Rails 1.2+
- Estándares basados en REST
 - Atom Publishing Protocol
 - WebDAV
 - Amazon S3
 - GData (basado en Atom Publishing Protocol)
 - OpenSearch (basado en Atom Publishing Protocol)

Conclusiones

- REST es un estilo arquitectural para aplicaciones distribuidas
- Introduce restricciones que producen propiedades deseables a cambio
 - Estas propiedades han permitido la expansión con éxito de la web
 - Las restricciones también pueden aplicarse a las aplicaciones y servicios web desarrollados para mantener las propiedades
- Los protocolos de la web, HTTP y URL, facilitan el desarrollo de arquitecturas RESTful y permiten explotar sus propiedades
- REST no es el único estilo arquitectural
 - Es posible eliminar restricciones si no nos importa perder propiedades
 - Hay otros estilos arquitecturales con otras propiedades. Ejemplo: RPC

Referencias

- “Chapter 5: Representational State Transfer (REST)”, Fielding, Roy Thomas, en “Architectural Styles and the Design of Network-based Software Architectures” (PhD thesis) University of California, Irvine 2000.
- “RESTful Web Services”, Leonard Richardson, Sam Ruby, O’Reilly Press
- “Architectural Styles and the Design of Network-based Software Architectures” (Capítulo 5)
http:
[//www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm](http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm)
- “Best Practices for Designing a Pragmatic RESTful API”, Vinay Sahni
http:
[//www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api](http://www.vinaysahni.com/best-practices-for-a-pragmatic-restful-api)
- Atom Publishing Protocol:
<http://tools.ietf.org/html/rfc5023>

Arquitectura modelo-vista-controlador

¿Qué es la arquitectura MVC?

- Patrón de arquitectura (implementación)
- Desacopla tres elementos:
 - datos (estado de la aplicación)
 - representación en la interfaz
 - lógica de aplicación
- Modelo: datos (estado) y su gestión
- Vista: Representación en la interfaz (filtrado, actualización de datos)
- Controlador: lógica de la aplicación y flujo de información

MVC en aplicaciones web (1)

The model is any of the logic or the database or any of the data itself. The view is simply how you lay the data out, how it is displayed. [...]

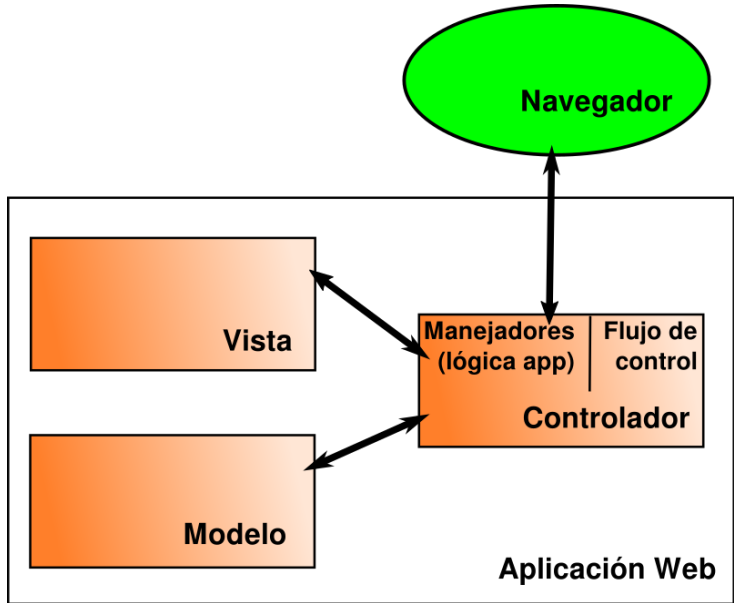
The controller in a web app is a bit more complicated, because it has two parts. The first part is the web server (such as a servlet container) that maps incoming HTTP URL requests to a particular handler for that request. The second part is those handlers themselves, which are in fact often called “controllers”. [...]

“The Importance of Model-View Separation”, Terence Parr

MVC en aplicaciones web

- Modelo:
Descripción y gestión de base de datos
- Vista:
Interfaz de usuario (HTML)
que presenta el modelo al usuario
- Controlador:
Recibe indicaciones del usuario,
indica cambios al modelo,
elige vista para mostrar resultados

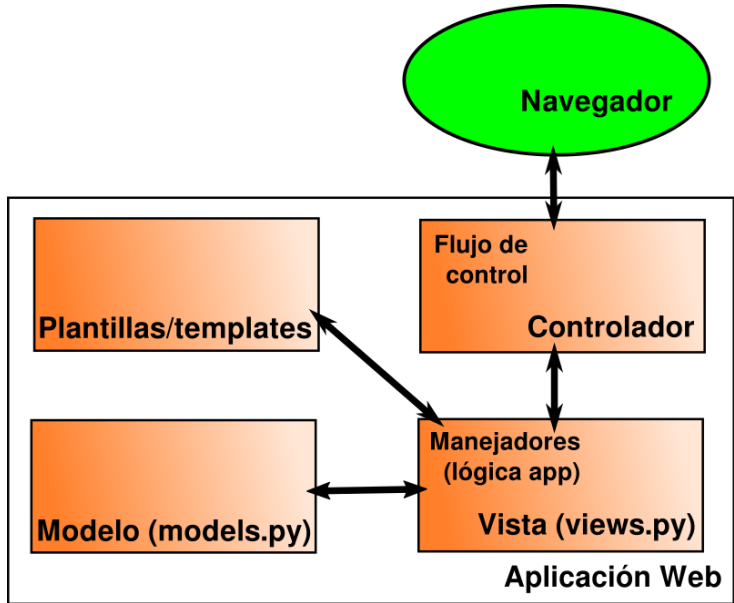
MVC en aplicaciones web

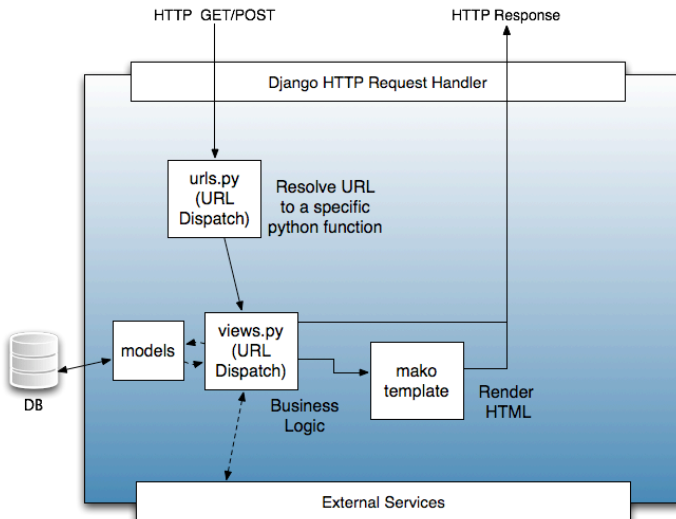


MVC en Django

- Vista: Manejadores (lógica de aplicación) funciones en `views.py`
- Vistas delegan la presentación en plantillas (templates)
- Modelo: Descripción de los datos clases en `models.py`
- Controlador: maquinaria de Django incluyendo `urls.py`
- Django como “plataforma MTV”: modelo, plantilla (template), vista

Django: MTV





Fuente: <http://archive.cloudera.com/cdh4/cdh/4/hue/sdk/sdk.html>

Referencias

- MVC, Xerox PARC 1978-79:
`http://heim.ifi.uio.no/~trygver/themes/mvc/mvc-index.html`
- “The Importance of Model-View Separation. A Conversation with Terence Parr”, by Bill Venner
`http://www.artima.com/lejava/articles/stringtemplate.html`

Créditos, licencia...

Creditos



Derivado de "Channapatna Toys", HPNadig
CC BY-SA 3.0, vía Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:Channapatna-toys.jpg>



©2002-2026 Jesús M. González Barahona, Gregorio Robles,
David Moreno Lumbreras, Jorge Ferrer y Francisco Servant.

Algunos derechos reservados. Este artículo se distribuye bajo la licencia
“Reconocimiento-CompartirIgual 4.0 Internacional” de Creative Commons, disponible en
<http://creativecommons.org/licenses/by-sa/4.0/es/deed.es>

Este documento (o uno muy similar) está disponible en
<http://cursosweb.github.io>